

ragent Tutorial: From Setup to Release

This tutorial is a hands-on, end-to-end walkthrough for using **ragent** through its full-screen terminal UI. It covers the complete lifecycle of a project: selecting an agent mode, setting up a new project, generating code, debugging, and releasing to a version-control host such as GitHub.

Each section includes step-by-step instructions and copy-paste-ready prompts. Where another how-to document covers a topic in greater depth, it is cross-referenced so you can go deeper without re-reading material here.

Table of Contents

1. Prerequisites
 2. Starting the TUI
 3. Selecting a Mode (Agent Preset)
 4. Setting Up a New Project
 5. Generating Code for the Project
 6. Debugging the Project
 7. Releasing the Project to GitHub
 8. Keybindings Quick Reference
 9. Related How-To Documents
-

1. Prerequisites

Before you begin, make sure you have:

- **ragent** built and on your PATH (see `README.md` for build instructions).
- At least one LLM provider configured. **ragent** supports Anthropic, OpenAI, Google Gemini, Hugging Face, GitHub Copilot, Ollama (local and cloud), Generic OpenAI-compatible endpoints, Azure AI Foundry, Azure Resource, Amazon Bedrock, Microsoft Foundry Local, OpenRouter, XAI, and a Model Router cluster. For a fully local setup, install Ollama and pull a model such as `qwen2.5-coder:7b`.
- A terminal supporting 256 colours and Unicode.
- Git on your PATH if you intend to release to a VCS.

You do not need a project directory yet — the next section starts from a clean workspace.

Selecting a provider and model

If this is your first run, the TUI opens a provider setup dialog. Type `/provider` at any time to change providers:

1. Select a provider from the list.
2. Enter your API key (or skip for local providers like Ollama).
3. Choose a model from the provider's model list.
4. Press `Enter` to confirm.

The status bar shows the active provider, model, and a health indicator: green = reachable, yellow = checking, red = unreachable or key missing.

```
ollamacloud/kimi-k2.7-code green
```

Use `/model` to switch models within the current provider. For the full provider table, see `TUI-QUICKSTART.md`.

2. Starting the TUI

Open a terminal and navigate to the directory where you want your new project to live (or create one):

```
mkdir -p ~/Projects/my-app
cd ~/Projects/my-app
ragent
```

The TUI opens with three main regions:

- **Status bar** (top) — active provider, model, agent, and health indicator.
- **Message pane** (centre) — conversation with the agent, including tool calls.
- **Input panel** (bottom) — where you type prompts and slash commands.

Useful startup flags

Flag	Purpose
<code>--model ollamacloud/kimi-k2.7-code</code>	Start with a specific model
<code>--agent coder</code>	Start with the coder agent profile
<code>--yes</code>	Auto-approve all permission prompts
<code>--no-tui</code>	Run a single prompt without the TUI
<code>--log</code>	Open with the log panel already visible

```
ragent --model ollamacloud/kimi-k2.7-code --agent coder
```

For a detailed walkthrough of the TUI layout and panels, see `TUI-QUICKSTART.md` in the project root.

3. Selecting a Mode (Agent Preset)

`ragent` ships with several built-in agent presets, each tuned to a specific workflow. The preset controls the system prompt, default permissions, and tool emphasis. Switch presets at any time without restarting.

Built-in presets

Preset	Best for
<code>general</code>	General-purpose coding and Q&A (default)
<code>coder</code>	Hands-on code writing and refactoring
<code>ask</code>	Quick read-only questions about the codebase
<code>debug</code>	Diagnosing failures, panics, and test failures
<code>architect</code>	High-level design and planning
<code>code-review</code>	Reviewing diffs and pull requests

Preset	Best for
task	Focused, single-objective tasks
orchestrator	Coordinating multiple sub-tasks or sub-agents

Selecting a preset in the TUI

Type `/agent` and press `Enter` to open the interactive picker. Use the arrow keys to highlight a preset, then press `Enter` to confirm. Or switch directly:

```
/agent coder
```

The status bar updates immediately and the Log panel (`Alt+L`) records the switch.

Creating custom agents

If no built-in preset fits, define your own via Markdown profiles (`.md`) or OASF JSON (`.json`). Place files in `.ragent/agents/` (project-local) or `~/.ragent/agents/` (user-global). For the full schema and examples, see [docs/howtos/custom-agents.md](#).

4. Setting Up a New Project

This section walks through creating a new Rust CLI project, configuring the workspace, and establishing project guidelines that `ragent` follows for the rest of the session.

4.1 Initialise the project skeleton

Switch to the `coder` agent:

```
/agent coder
```

Ask `ragent` to scaffold the project:

Create a new Rust CLI project in the current directory. It should be a binary crate called "my-app" with:

- A `main.rs` entry point
- A `lib.rs` module exposing public functions
- A `modules/` subdirectory with placeholder modules for `cli`, `config`, and `commands`
- A `Cargo.toml` with `anyhow`, `clap`, `serde`, and `tokio` as dependencies
- A `tests/` directory with a smoke test
- A `.gitignore` for the `target/` directory

`ragent` calls `bash` to run `cargo init`, then `write` and `edit` to create the files. When it requests permission, respond with `y` (allow once) or `a` (allow always for this session). To skip prompts entirely, press `Alt+Y` to toggle YOLO mode, or start with `--yes`.

4.2 Create an `AGENTS.md` guidelines file

`AGENTS.md` is a project-level instruction file loaded into the system prompt on every launch. Ask `ragent` to generate one:

Create a concise AGENTS.md for this Rust project covering the technology stack, code style, testing conventions, and tool preferences.

A typical result:

```
# Project Guidelines

## Technology Stack
- Language: Rust (edition 2024)
- Build tool: Cargo

## Code Style
- 4-space indentation, 100-column line limit
- snake_case for functions, PascalCase for types
- Prefer Result<T, E> error handling

## Testing
- Tests in each crate's tests/ directory
- #[test] for sync, #[tokio::test] for async

## Tool Preferences
- Use codeindex_search for code symbol lookups
- Prefer multi_edit when changing several files at once
```

Keep it concise — ragent reads it on every launch.

4.3 Enable the code index

The code index provides fast, structured search using tree-sitter parsing and Tantivy full-text search. Turn it on:

```
/codeindex on
```

Verify status:

```
/codeindex show
```

The agent can now use `codeindex_search`, `codeindex_references`, `codeindex_dependencies`, and `codeindex_symbols` instead of `grep` for symbol queries. For advanced graph analysis (god-nodes, shortest paths, community detection):

```
/codeindex graph build
```

4.4 Initialise git and enable GitHub tools

Initialise a git repository, create an initial commit with all scaffold files, and set the remote to `git@github.com:myuser/my-app.git`

If GitHub tools are not visible, enable them:

```
/tools github on
```

This persists to `ragent.json`. See `docs/howtos/tool-visibility.md` for the full list of switches and defaults.

4.5 Authenticate with GitHub

```
/github login
```

A dialog appears with a verification URL and a user code. Open the URL in a browser, enter the code, and authorise. ragent polls in the background and stores the token in `~/.ragent/github_token`.

Check status:

```
/github status
```

For GitLab, the flow is analogous: `/gitlab setup`.

5. Generating Code for the Project

With the scaffold in place, the code index active, and the coder agent selected, ask ragent to implement features.

5.1 Implementing a feature

Describe what you want in plain language, specifying file locations and expected behaviour:

Implement a "greet" subcommand in `src/cli.rs`. It should accept a `--name` argument (default "World") and print "Hello, {name}!" to stdout. Add a unit test in `tests/` that verifies the output.

ragent reads the existing file (via `read` or `codeindex_search`), edits `src/cli.rs` (via `edit` or `multi_edit`), creates the test file, and optionally runs `cargo check` to verify. Each tool call appears in the message pane with a step number; the Log panel shows full JSON.

5.2 Bang commands for quick shell access

Prefix any prompt with `!` to run a shell command and have the model review the output:

```
! cargo test -- --nocapture
```

5.3 Using specs for structured features

For larger features, create a formal specification first:

```
/spec create add-config-file Add a TOML config file loader with  
environment variable overrides
```

ragent generates `SPEC.md`, `PLAN.md`, and `TESTPLAN.md` under `specs/add-config-file/`. Validate, then implement by asking ragent to run the spec's implementation command (e.g. `/spec impl add-config-file` to execute the plan's tasks in dependency order, or `/spec impl add-config-file --dry-run` to preview it):

```
/spec validate add-config-file  
/spec impl add-config-file
```

Track tasks:

```
/spec task add-config-file T-001 completed
```

See docs/howtos/spec.md for the full command reference and SDD workflow.

5.4 Using research for unfamiliar domains

```
/research create toml-config "TOML configuration file patterns in Rust"
```

ragent searches the web, fetches key pages, cross-references local files, and writes research/toml-config/RESEARCH.md. Link research to a spec:

```
/spec create config-loader "Implement TOML config loading" --from-research toml-config
```

See docs/howtos/research.md for tiers, output formats, and the full command family.

5.5 Generating code from a reference repository

```
/reverse BurntSushi/toml-rs --tech rust
```

Chain into spec creation:

```
/reverse BurntSushi/toml-rs --create toml-clone
```

See docs/howtos/reverse.md for the full command syntax and flags.

5.6 Parallel work with teams

For features split into independent streams (API, UI, tests), use teams:

```
/team create feature-squad
```

ragent spawns teammates from a blueprint with role-specific prompts. The lead coordinates via a shared task queue and mailbox messaging. See docs/howtos/teams.md for the full manual.

6. Debugging the Project

When the build breaks or tests fail, switch to the debug agent.

6.1 Switch to the debug agent

```
/agent debug
```

The debug preset is tuned for failure analysis: it prioritises reading logs, reproducing errors, and making minimal targeted fixes.

6.2 Reproduce the failure

Run "cargo test -- --nocapture" and diagnose any failures. Fix the root cause and re-run the tests to confirm.

Or use a bang command for a quick check:

```
! cargo test 2>&1 | tail -40
```

6.3 Reading errors in the Log panel

Press `Alt+L` to open the Log panel. Colour-coded events:

Prefix	Meaning
INF	Information
TUL	Tool call / result
WRN	Warning
ERR	Error
CMP	Compaction event

Click a log line to start a text selection; copy with `Ctrl+C`.

6.4 Using the Profile panel for slow operations

Press `Alt+P` to open the Profile panel. It shows uptime, samples, and operations sorted by self time. Use it to spot slow tool calls or repeated work.

6.5 Using the code index to trace dependencies

Find all call sites of the `parse_config` function and update them to pass the new `Config` struct by reference instead of by value.

The agent uses `codeindex_references` to find every call site, then `multi_edit` to update them atomically.

6.6 Inspecting git history

Show me the last 5 commits and the diff for `src/cli.rs` since `HEAD~3`. Identify which commit introduced the failing test.

ragent calls `git_log` and `git_diff` to gather the history and analyse it.

6.7 Creating a task list for the fix

Create a task list for fixing the config loader:

1. Reproduce the panic with a minimal test case
2. Identify the root cause in `src/config.rs`
3. Fix the parsing logic
4. Add a regression test
5. Verify all tests pass

Open the TASKS panel (`Alt+T`) to watch progress in real time.

7. Releasing the Project to GitHub

Once the code is working and tested, release it to your VCS.

7.1 Enable GitHub tools and authenticate

```
/tools github on  
/github login
```

7.2 Review the working tree

Show me the current git status and a summary of the diff.

ragent calls `git_status` and `git_diff --stat`.

7.3 Commit changes

Stage all changes and commit with the message "Add greet subcommand with unit tests".

To review the staged diff first:

Show me the staged diff before committing.

7.4 Create and push a feature branch

Create a new branch called "feature/greet-command", push it to the origin remote, and set up upstream tracking.

ragent uses `git_checkout` and `git_push`.

7.5 Create a pull request

Create a pull request titled "Add greet subcommand" with a body describing the changes. Set the base branch to "main".

ragent calls `github_create_pr`. The head branch must already be pushed.

7.6 Review and merge

Show me the open pull requests and the latest GitHub Actions runs.

ragent calls `github_list_prs` and `github_get_actions`. Once CI passes:

Merge the pull request using the squash method.

ragent calls `github_merge_pr` with method: "squash".

7.7 Tag a release

Update the version in `Cargo.toml` to 0.1.0, add a changelog entry, commit with "Version: 0.1.0", push to main, and tag the commit as v0.1.0.

ragent edits `Cargo.toml` and `CHANGELOG.md`, commits, pushes, creates the tag, and pushes the tag.

7.8 Create a GitHub release

Use the gh CLI through a bang command:

```
! gh release create v0.1.0 --title "v0.1.0" --notes "Initial release"
```

7.9 Automating the release flow

Create a custom agent in `.ragent/agents/release.md`:

```
---
{
  "name": "release",
  "description": "Release agent that bumps versions and tags releases"
}
---
```

You are a release agent. When asked to release:

1. Read the current version from `Cargo.toml`.
2. Increment the patch version by 1.
3. Update `Cargo.toml` and `CHANGELOG.md`.
4. Commit with "Version: `<new-version>`".
5. Push to main and tag as `v<new-version>`.
6. Push the tag.

Report the old and new versions.

Then:

```
/agent release
```

Release the project with the message "Add greet subcommand and tests"

See `docs/howtos/custom-agents.md` for the full schema.

8. Keybindings Quick Reference

Key	Action
Enter	Send prompt
Shift+Enter / Alt+Enter	New line in input
Escape	Cancel current agent operation
Ctrl+D	Quit ragent
Ctrl+C	Copy selection / arm quit
Alt+L	Toggle Log panel
Alt+P	Toggle Profile panel
Alt+T	Toggle TASKS panel
Alt+M	Toggle Memory panel
Alt+O	Toggle Telemetry panel
Alt+C	Toggle Context side panel
Alt+V	Paste image from clipboard
Alt+Y	Toggle YOLO mode

Key	Action
@	Open file mention picker
/	Open slash-command menu
?	Show keybindings help

Log and Profile can coexist (Log above, Profile below). Other side panels are mutually exclusive. All support mouse scrolling and scrollbar dragging. Every toggled side panel (Log, Profile, TASKS, Memory, Telemetry, and Context) takes 50% of the application width.

9. Related How-To Documents

Document	Covers
TUI-QUICKSTART.md	Full TUI layout, panels, startup options
docs/howtos/custom-agents.md	Custom agent profiles and OASF schema
docs/howtos/teams.md	Multi-agent team coordination
docs/howtos/spec.md	Spec management and SDD workflow
docs/howtos/research.md	Research system and report synthesis
docs/howtos/reverse.md	Repository reverse-engineering
docs/howtos/tool-visibility.md	Hiding and exposing tool families
docs/howtos/communications.md	Gmail and messaging channel tools
docs/howtos/finance.md	Stock and currency tools

For topics not covered here — MCP integration, autopilot mode, prompt optimization, or the HTTP server — see QUICKSTART.md and SPEC.md in the project root, or run `/help` inside the TUI.